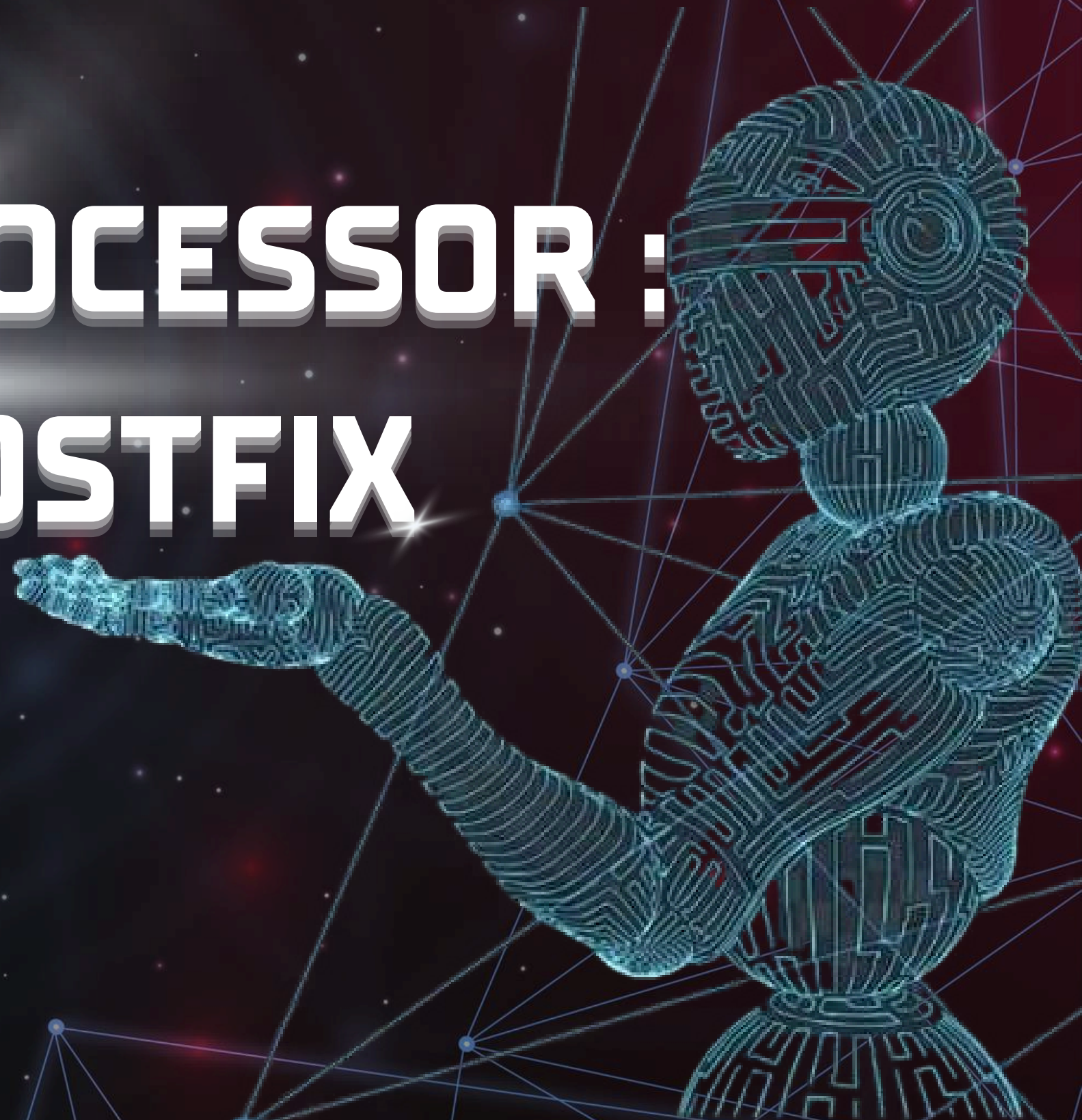


EXPRESSION PROCESSOR : INFIX $\leftrightarrow$ POSTFIX

Group 4



สมาชิก

นาย กษิธิศ อุ่นอำไพ	68122250028
นาย ชลสิทธิ์ จิตรณรงค์	68122250092
นางสาว น้ำผึ้ง เจริญดวงค์	68122250031
นาย ณัฐชนันท์ จินดาณิล	68122250102
นาย พฤทธชาต ดำนิล	68122250104

CASE STORY

ระบบรับนิพจน์ทางคณิตศาสตร์ ตรวจสอบความถูกต้อง แปลง Infix เป็น Postfix และคำนวณผลลัพธ์ พร้อมเปรียบเทียบกับวิธีประเมิน Infix โดยตรงด้วย Operand Stack และ Operator Stack



INPUT/OUTPUT/CONSTRAINTS

- Input** : Infix Expression
- Output** : Postfix (สำหรับ A), Result, Error ถ้ามี
- Constraint** : Operator เป็น binary operator; `\*`,`/` priority 2; `+`,`-` priority 1; ทำจากซ้ายไปขวาเมื่อ priority เท่ากัน

Algorithm A

แนวคิด

1. Tokenize และ Validate
2. Infix $\rightarrow$ Postfix ด้วย Operator Stack
3. Evaluate Postfix ด้วย Operand Stack

Algorithm B

แนวคิด

ใช้ Operand Stack และ Operator Stack
ประเมิน Infix โดยตรง โดยเปรียบเทียบ
Priority และคำนวณเมื่อถึงจุดที่ควรทำ

Pseudocode Algorithm A

```
INPUT tokens
CREATE OperatorStack and Output
FOR each token
    NUMBER -> append to Output
    '(' -> push to OperatorStack
    ')' -> pop operators to Output until '('
    OPERATOR -> pop operators with priority >= current, then push current
END FOR
Pop remaining operators to Output
Evaluate Postfix using OperandStack
RETURN result
```

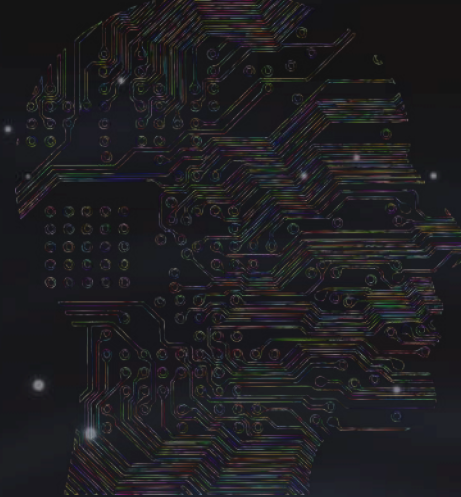
Algorithm B

```
INPUT tokens
CREATE OperandStack and OperatorStack
FOR each token
    NUMBER -> push OperandStack
    '(' -> push OperatorStack
    ')' -> apply operators until '('
    OPERATOR -> apply top operators with priority >= current, then push current
END FOR
Apply remaining operators
RETURN top of OperandStack
```


Step-by-step

Infix: $3 + 4 * 2 / (1 - 5)$

Postfix: $3\ 4\ 2\ *\ 1\ 5\ -\ /\ +$ **result:** 1



Step	Token	Operator Stack	Output
1	3	[]	3
2	+	[+]	3
3	4	[+]	3 4
4	*	[+, *]	3 4
5	2	[+, *]	3 4 2
6	/	[+, /]	3 4 2 *
7	(	[+, /, (]	3 4 2 *
8	1	[+, /, (]	3 4 2 * 1
9	-	[+, /, (, -]	3 4 2 * 1
10	5	[+, /, (, -]	3 4 2 * 1 5
11	)	[+, /]	3 4 2 * 1 5 -
12	จบ	[]	3 4 2 * 1 5 - / +

Step	Token	Operand Stack	การทำงาน
1	3	[3]	Push
2	4	[3, 4]	Push
3	2	[3, 4, 2]	Push
4	*	[3, 8]	$4 \times 2 = 8$
5	1	[3, 8, 1]	Push
6	5	[3, 8, 1, 5]	Push
7	-	[3, 8, -4]	$1 - 5 = -4$
8	/	[3, 2]	$8 \div -4 = -2$
9	+	[1]	$3 + (-2) = 1$

Correctness Argument

Precondition

- ก่อนเริ่ม Algorithm
- Input มี Token ตามรูปแบบที่ระบบรองรับ
 - Operand เป็นจำนวนเต็มบวก
 - Operator อยู่ใน + - * /
 - วงเล็บต้องสมดุล
 - นิพจน์ต้องมีลำดับ Operand และ Operator ถูกต้อง

Loop Invariant

- ระหว่างการแปลง Infix เป็น Postfix มีคุณสมบัติที่ยังคงเป็นจริงดังนี้
- Operand ที่อ่านมาแล้วและไม่ต้องรอ Operator จะถูกเก็บใน Output
 - Operator ที่ยังไม่ควรถูกนำออกจะอยู่ใน Operator Stack
 - Operator Priority สูงกว่าจะถูกประมวลผลก่อน Operator Priority ต่ำกว่า
 - Operator ที่อยู่ในวงเล็บจะไม่ถูกนำออกไปนอกวงเล็บ

กรณีวงเล็บ

เมื่อพบ [ระบบ Push ลง Stack เพื่อเป็นขอบเขตของนิพจน์
เมื่อพบ] ระบบจะ Pop Operator จนถึง [ทำให้การคำนวณ
ภายในวงเล็บเสร็จก่อน

กรณี Operand

เมื่อพบ Operand ระบบนำ Operand ไป Output หรือ
Operand Stack โดยตรง เนื่องจาก Operand ไม่จำเป็นต้อง
เปรียบเทียบ Priority

กรณี Operator

เมื่อพบ Operator ระบบเปรียบเทียบ Priority กับ Operator บน Stack
หาก Operator บน Stack มี Priority มากกว่าหรือเท่ากัน ระบบจะ Pop
ออกมาก่อน ทำให้ Operator ที่ต้องคำนวณก่อนถูกประมวลผลตามลำดับ

Correctness ของ Postfix

Postfix ไม่ต้องใช้วงเล็บในการกำหนดลำดับ เพราะตำแหน่งของ
Operator ใน Postfix เป็นตัวกำหนดลำดับการคำนวณอยู่แล้ว

Correctness Argument

ตัวอย่าง

$3 + 4 * 2$

กลายเป็น

$3\ 4\ 2\ * +$

เมื่อประเมินจะทำ

$4 * 2$

ก่อน แล้วจึง

$3 + 8$

จึงได้

11

Postcondition

เมื่อ Algorithm หยุด

Operator ที่ต้องประมวลผลทั้งหมดถูกนำออกจาก Stack

Operand Stack เหลือผลลัพธ์สุดท้ายเพียงหนึ่งค่า

ผลลัพธ์ตรงกับค่าของนิพจน์ Infix



ดังนั้น Algorithm ให้ผลลัพธ์ที่ถูกต้อง



Big-O Analysis

ให้ n คือจำนวน Token ของนิพจน์

Algorithm A

Tokenize

$O(n)$

Infix $\rightarrow$ Postfix

$O(n)$

แม้จะมี while อยู่ภายใน Loop แต่ Operator แต่ละตัว ถูก Push และ Pop จำนวนจำกัด จึงรวมแล้วเป็นเชิงเส้น

Evaluate Postfix

$O(n)$

ดังนั้น Algorithm A โดยรวม

$O(n)$

Algorithm B

Algorithm B อ่าน Token และทำ Push/Pop Stackแบบจำกัดดังนั้น $O(n)$

Algorithm	Best	Average	Worst
Algorithm A	$O(n)$	$O(n)$	$O(n)$
Algorithm B	$O(n)$	$O(n)$	$O(n)$



Big-O Analysis

Space Complexity

Algorithm A

Operator Stack = $O(n)$

Postfix Output = $O(n)$

Operand Stack = $O(n)$

ดังนั้น

Auxiliary Space = $O(n)$

Algorithm B

Operator Stack = $O(n)$

Operand Stack = $O(n)$

ดังนั้น

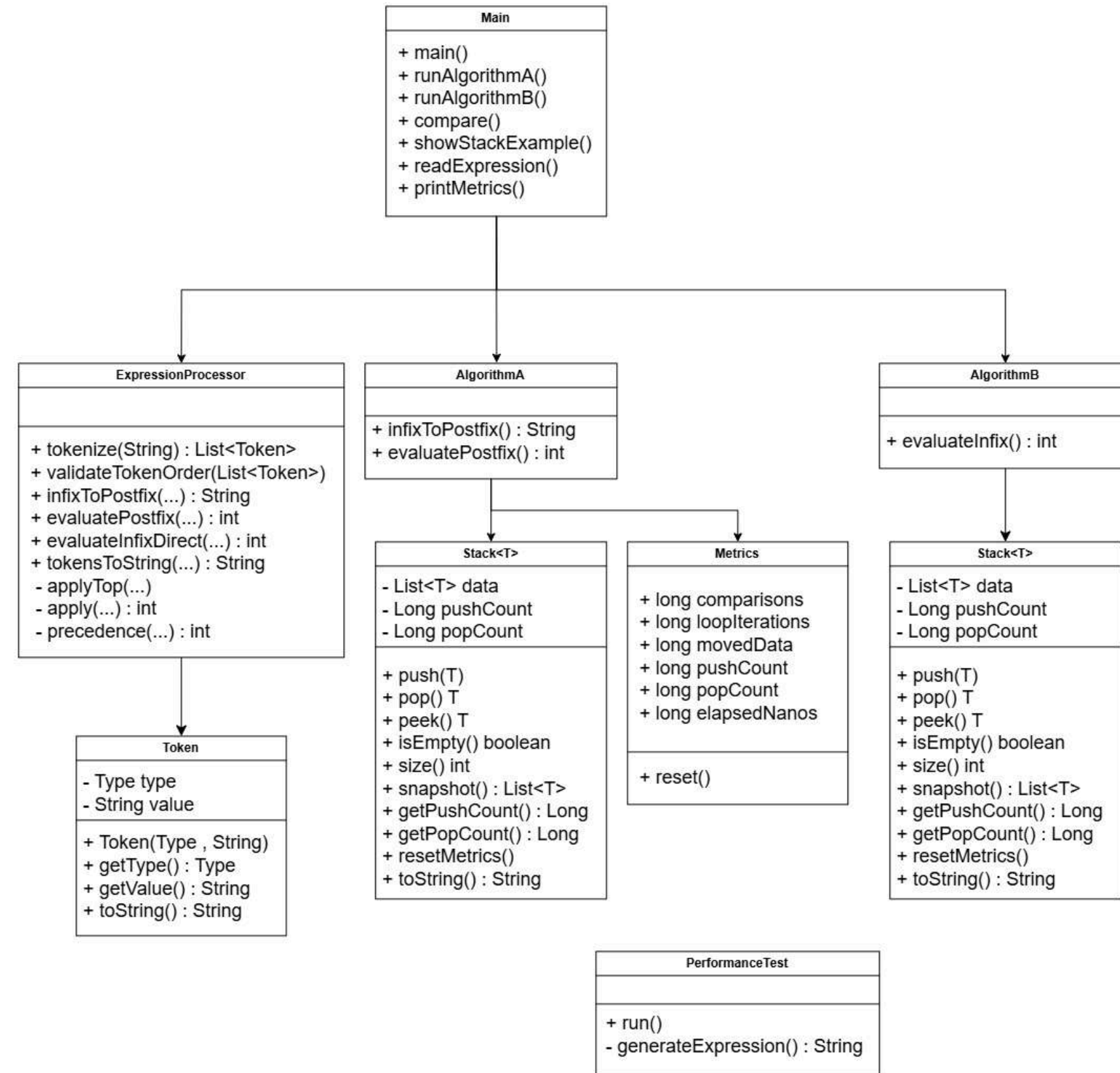
Auxiliary Space = $O(n)$

Big-Omega และ Big-Theta

ทั้งสอง Algorithm ต้องอ่าน Input เพื่อประมวลผล ดังนั้น $\Omega(n)$

และภายใต้โมเดลการคำนวณของโจทช์ $\Theta(n)$

Class Diagram



Java Implementation

โครงสร้างหลัก

```
src/  
├─ Main.java  
├─ models/  
│   └─ Token.java  
├─ algorithms/  
│   └─ ExpressionProcessor.java  
├─ utils/  
│   ├── MyStack.java  
│   └─ Metrics.java  
└─ experiments/  
    └─ PerformanceTest.java
```

Main Class

```
public class Main {  
    public static void main(String[] args) {  
        // แสดง Menu  
        // รับ Expression  
        // เลือก Algorithm A หรือ B  
        // แสดงผลลัพธ์  
    }  
}
```


Java Implementation

Data Model

```
public class Token {  
    public enum Type {  
        NUMBER,  
        OPERATOR,  
        LEFT_PAREN,  
        RIGHT_PAREN  
    }  
  
    private final Type type;  
    private final String value;  
  
    public Token(Type type, String value) {  
        this.type = type;  
        this.value = value;  
    }  
  
    public Type getType() {  
        return type;  
    }  
  
    public String getValue() {  
        return value;  
    }  
}
```

Stack

```
public class MyStack<T> {  
  
    private final List<T> data = new ArrayList<>();  
  
    public void push(T item) {  
        data.add(item);  
    }  
  
    public T pop() {  
        if (isEmpty()) {  
            throw new IllegalStateException("Stack is empty");  
        }  
  
        return data.remove(data.size() - 1);  
    }  
  
    public T peek() {  
        if (isEmpty()) {  
            throw new IllegalStateException("Stack is empty");  
        }  
  
        return data.get(data.size() - 1);  
    }  
  
    public boolean isEmpty() {  
        return data.isEmpty();  
    }  
}
```


Test case

Test Case Table

ID	Input	Expected	Case
TC01	3 + 4 * 2	11	Normal / Priority
TC02	(3 + 4) * 2	14	Parentheses
TC03	((8 + 2) * 5)	50	Nested parentheses
TC04	(3 + 4	Error: วงเล็บไม่สมดุล	Boundary/Error
TC05	3 + 4)	Error: วงเล็บปิดเกิน	Boundary/Error
TC06	3 + * 4	Error: Operator อยู่ผิดตำแหน่ง	Validation
TC07	10 / (5 - 5)	Error: หารด้วยศูนย์ไม่ได้	Arithmetic Error
TC08	empty	Error: นิพจน์ว่าง	Boundary
TC09	100 / 5 / 2	10	Left associativity
TC10	12 + 34	46	Multi-digit operand
TC11	3 & 4	Error: อักขระไม่รองรับ	Invalid token

Experimental Results

	column 1	column 2	column 3	column 4	column 5	column 6	column 7	column 8 ↑ ₁
1	100	B	108253.2	991.0	197.0	298.0	297.0	199.0
2	100	A	812809.4	1190.0	197.0	298.0	297.0	398.0
3	1000	B	629617.6	9991.0	1997.0	2998.0	2997.0	1999.0
4	1000	A	2969527.0	11990.0	1997.0	2998.0	2997.0	3998.0
5	10000	B	869445.8	99991.0	19997.0	29998.0	29997.0	19999.0
6	10000	A	27551009.8	119990.0	19997.0	29998.0	29997.0	39998.0
7	50000	B	2291715.6	499991.0	99997.0	149998.0	149997.0	99999.0
8	50000	A	90035339.8	599990.0	99997.0	149998.0	149997.0	199998.0
9	n	algorithm	avg_time_ns	avg_operations	avg_comparisons	avg_push	avg_pop	avg_loops

สรุปและวิธีที่กลุ่มเลือก

จากการออกแบบและเปรียบเทียบ Algorithm ทั้งสอง วิธีสามารถประมวลผลนิพจน์ตามโจทย์ได้อย่างถูกต้อง และทั้งสองวิธีมี Time Complexity เป็น $O(n)$ และ Auxiliary Space เป็น $O(n)$

Algorithm A: Infix $\rightarrow$ Postfix $\rightarrow$ Evaluate

มีข้อดีคือโครงสร้างชัดเจน เข้าใจง่าย และสามารถตรวจสอบลำดับการคำนวณผ่าน Postfix ได้ง่าย จึงเหมาะสำหรับการอธิบายและตรวจสอบความถูกต้องของระบบ

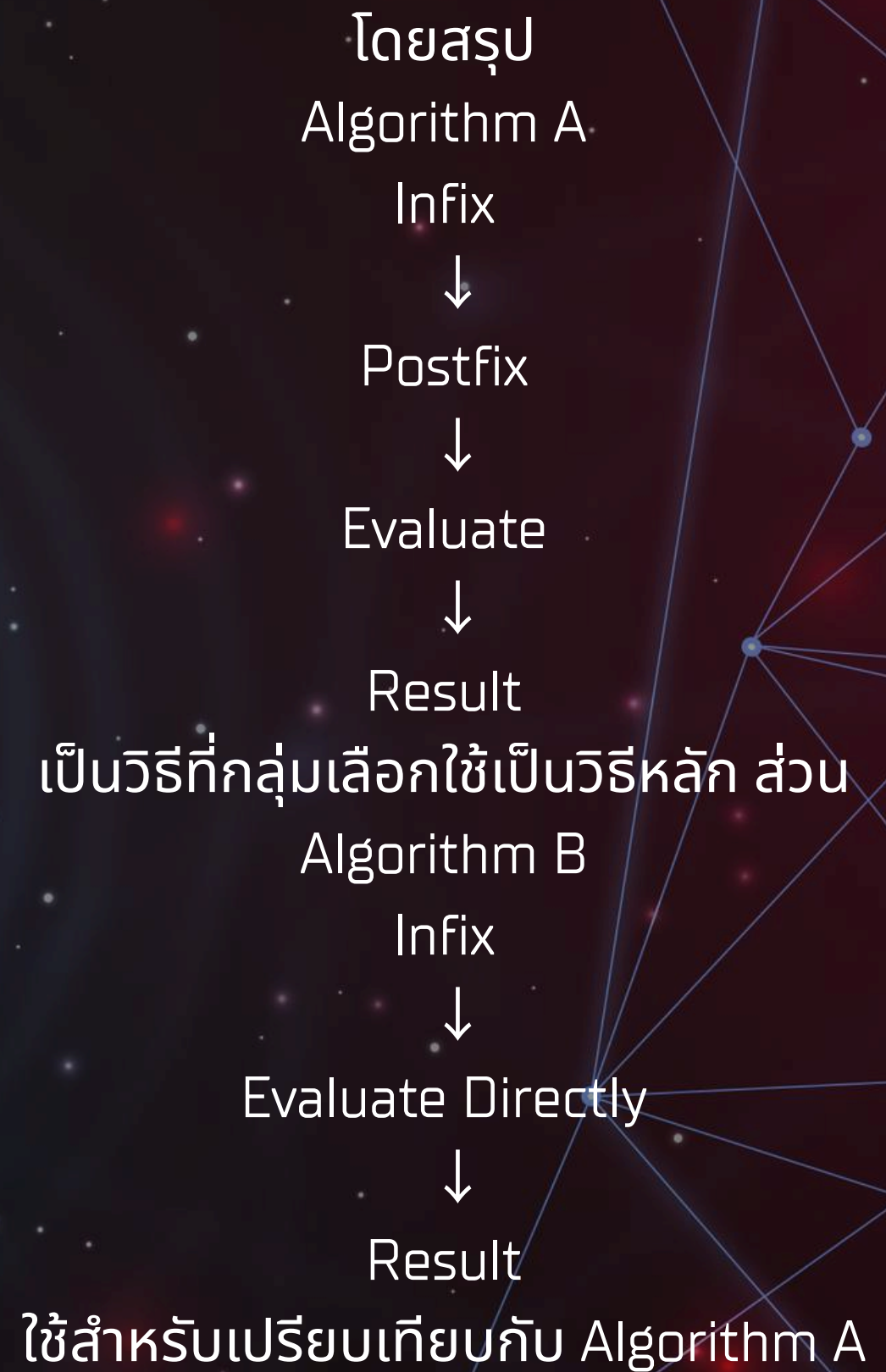
Algorithm B: Direct Infix Evaluation

มีข้อดีคือสามารถประเมินนิพจน์ได้โดยตรงและไม่ต้องสร้าง Postfix แยก แต่ Logic ในการจัดการ Priority และ Stack มีความซับซ้อนมากกว่า

วิธีที่กลุ่มเลือก

กลุ่มเลือก Algorithm A: Infix $\rightarrow$ Postfix $\rightarrow$ Evaluate เป็นวิธีหลักในการนำเสนอ เนื่องจากมีขั้นตอนที่ชัดเจน เข้าใจง่าย และแสดงลำดับการคำนวณได้อย่างชัดเจนผ่าน Postfix อีกทั้งยังเหมาะสำหรับการอธิบายเรื่อง Stack, Operator Priority และ Correctness ของอัลกอริทึม

อย่างไรก็ตาม กลุ่มยังพัฒนา Algorithm B เพื่อใช้เป็นวิธีเปรียบเทียบกับประสิทธิภาพ ตามข้อกำหนดที่ต้องออกแบบอัลกอริทึมสำหรับปัญหadeียวกันมากกว่า 1 วิธี



จบการนำเสนอ